Exception Events Are All You Need: Fault Indicator Isolation as the Robust Mechanism for Execution Trace Verdict Prediction

Anonymous

Abstract

Execution traces have been used to improve code understanding through supervised fine-tuning (SFT), but prior work finds inconsistent, model-dependent benefits on generative tasks. We study a different question: what is the minimal sufficient trace signal for *verdict prediction*, the binary classification of whether code passes or fails a test? Through systematic ablation across four architecturally distinct models (Qwen-2.5-Coder-7B, Qwen-2.5-Coder-1.5B, DeepSeek-Coder-6.7B, CodeLlama-13B-Instruct), we find that **exception events** (compact, structured error summaries) are the only universally robust trace component, achieving 99.83% accuracy on Qwen-7B ($\pm$0.00 across 3 seeds) and consistently outperforming all alternatives on every model. All other trace components and budget-length relationships are model-dependent: four models produce four distinct budget curves with no shared phase transition pattern. SFT with exception-only traces achieves 99.83% accuracy at \$0.03 per 1,190 samples with near-perfect calibration (ECE = 0.0027), compared to GPT-4o few-shot prompting at 97.2% accuracy and \$15 per batch. The design principle is simple: for verdict tasks, extract the highest-specificity diagnostic signal and discard everything else.

1 Introduction

Execution traces record the runtime behavior of programs and contain rich information about control flow, variable states, and error conditions. Recent work has explored using traces to improve code language models through supervised fine-tuning. DCSH [1] tested five trace formats on generative tasks and found that "no single trace representation consistently outperforms others." TracePile [2] scaled trace SFT to 2.6 million samples but evaluated only generation tasks. TRACED [3] demonstrated that traces help encoder-based classification. Despite this growing body of work, two fundamental questions remain unanswered: (1) do traces help *verdict prediction* (binary pass/fail classification), and (2) what specific trace components drive classification performance?

These questions matter for practical software engineering. Verdict prediction is a core subroutine in automated testing, program repair, and code evaluation pipelines. If traces help, knowing *which* trace component matters determines the cost and complexity of the required instrumentation. A system that needs only exception events is radically simpler than one that requires full execution traces with loop unrolling and variable tracking.

We approach verdict prediction as **fault detection** rather than execution simulation. This reframing suggests that the most useful trace signal may not be the richest representation but the one with the highest specificity for faults. We test this hypothesis through systematic component ablation of VOTA (Verdict-Optimized Trace Abstraction), a trace compression pipeline that isolates three component types: exception summaries, assertion-point information, and loop compression patterns.

Our experiments across four models reveal a clear finding: exception events are the only universally robust trace component. On Qwen-7B, exception-only SFT achieves 99.83% accuracy with zero variance across three random seeds, and the two remaining errors are provably irreducible (caused by Python's `GeneratorExit` mechanism). On all four models, exception-only traces outperform every alternative representation. In contrast, all other findings are model-dependent. Budget curves (accuracy as a function of trace length) show four distinct patterns across four models: gradual monotonic (Qwen-7B), sharp delayed transition (Qwen-1.5B), early onset with tail regression (DeepSeek), and non-monotonic W-shape (CodeLlama). No universal phase transition exists.

We also compare against frontier model baselines. GPT-4o with 3-shot prompting reaches 97.2% accuracy on exception traces, close to but significantly below SFT (99.83%, $p = 0.001$). SFT's practical advantages are cost (300× cheaper), calibration (ECE $= 0.0027$ vs. no logit access), and offline deployment without API dependency.

Our contributions are:

1. We establish that exception events are the most effective and only universally robust trace component for verdict prediction, achieving near-perfect accuracy across four models while all other components show model-dependent behavior.
2. We quantify the information density bottleneck: label-only loss ablation shows gradient dilution accounts for only 26% of the full-trace-to-VOTA gap, while information overload accounts for 74%.
3. We reveal a two-tier diagnostic hierarchy where Tier 1 signals (exceptions, assertion-points) are individually near-perfect and sufficient, while Tier 2 signals (loop patterns) are model-dependent and introduce noise when combined with Tier 1.

2 Related Work

Trace-augmented code models. DCSH [1] introduced five trace representation formats for SFT and found that no format consistently outperforms others on generative tasks, with effectiveness varying across models and tasks. TracePile [2] scaled trace SFT to 2.6M samples and observed benefits for code generation, but did not study classification tasks. TRACED [3] used traces for encoder-based test verdict classification and showed improvements over code-only baselines, but did not perform component-level ablation to identify which trace elements are responsible. Our work differs in two ways: we focus on decoder-based SFT for binary verdict prediction, and we perform fine-grained ablation to isolate the active ingredient.

Code execution prediction. CRUXEval [4] introduced benchmarks for predicting execution outputs and inputs, measuring a model's ability to simulate program execution. This execution reasoning capability correlates with trace utilization effectiveness in our cross-model analysis. CodeExecutor [5] trained models to generate execution traces. Our work is complementary: rather than training models to produce traces, we study which trace components are most useful when consumed as input for classification.

LLM-based test verdict prediction. Prior approaches to automated verdict prediction include rule-based systems using test output parsing and neural approaches using code embeddings. Recent work has explored using large language models for code correctness assessment, typically through prompting. Our few-shot baseline experiments with GPT-4o and Claude situate SFT-based approaches within this landscape.

3 Method

3.1 Task Definition

Given a programming problem p, a candidate solution s, a test suite T, and optionally an execution trace τ produced by running s on T, the verdict prediction task is to predict a binary label $y \in \{\texttt{pass}, \texttt{fail}\}$ indicating whether s passes all tests in T.

3.2 Dataset

We construct our dataset from HumanEval (164 problems) and MBPP-sanitized (427 problems), totaling 591 unique programming problems. For each problem, we collect 10 LLM-generated candidate solutions, producing 5,910 samples. The pass/fail ratio is 71.5%/28.5%. We split into 4,720 training and 1,190 test samples with zero train-test input overlap. Execution traces are collected by instrumenting Python's tracing mechanism to record function calls, variable assignments, loop iterations, exceptions, and assertion results.

3.3 Trace Representations

We compare five trace conditions:

No-trace baseline. The model receives only the problem description p and candidate solution s, with no execution information.

Full trace. The model receives the complete execution trace τ, including all loop iterations, variable states, function calls, and exceptions. Traces are tail-truncated to fit the model's context window.

VOTA (Verdict-Optimized Trace Abstraction). A three-component compression of full traces:
- **Loop compression**: Repeated loop iterations are collapsed into a summary showing the first iteration, pattern statistics, and the last iteration.
- **Exception summary**: Exception events are extracted and formatted as structured records: `[ERROR] type: ... message: ...` or `[No exception]`.
- **Assertion-point information**: Actual vs. expected value comparisons at test assertion points.

Exception-only. Only the exception summary component of VOTA, discarding loop and assertion-point information.

Component ablations. We additionally test assertion-point-only (`ae_only`), loop-only (`loop_only`), and VOTA without assertion-points (`no_ae`).

3.4 Training Setup

All models are fine-tuned using LoRA (rank 16, $\alpha = 32$) with the SFT objective. Training runs for 3 epochs with a learning rate of 2e-4 and batch size 8. We evaluate on four models: Qwen-2.5-Coder-7B, Qwen-2.5-Coder-1.5B, DeepSeek-Coder-6.7B, and CodeLlama-13B-Instruct. For robustness evaluation, we train with three random seeds (42, 123, 456) where compute permits.

Table 1: Verdict prediction accuracy across trace conditions (Qwen-2.5-Coder-7B, seed 42). Exception-only achieves the highest accuracy despite being the simplest representation.

Condition	Accuracy	Pass F1	Fail F1	ECE
exception_only	**99.83%**	**0.999**	**0.997**	**0.0027**
ae_only	99.92%	0.999	0.999	0.0004
VOTA (full)	99.41%	0.996	0.989	0.0054
no_ae	99.41%	0.996	0.989	0.0054
loop_only	85.63%	—	—	0.0831
label_only	86.22%	—	—	0.1107
full_trace	81.76%	—	—	0.0958
no_trace	75.13%	—	—	0.1715

4 Experiments

4.1 Exception Events Dominate All Trace Components

Table 1 presents the main comparison across trace conditions on Qwen-7B. Exception-only traces achieve 99.83% accuracy (2 errors out of 1,190 test samples), outperforming full VOTA (99.41%), full traces (81.76%), and the no-trace baseline (75.13%). The exception-only condition exceeds VOTA despite containing strictly less information ($p = 0.01$), indicating that non-exception components introduce noise rather than useful signal.

Component ablation reveals a strict two-tier diagnostic hierarchy. **Tier 1** consists of exception summaries (99.83%) and assertion-point information (99.92%), which are individually near-perfect and statistically equivalent ($p = 0.80$). **Tier 2** consists of loop compression patterns (85.63%), which carry substantial signal above baseline but are fully subsumed by Tier 1. Removing assertion-point information from VOTA does not change accuracy (99.41% with and without), confirming that the exception component is VOTA's active ingredient.

Irreducible error analysis. The 2 errors made by exception-only (sample ids 901 and 905) are identical across all 3 random seeds. Both come from HumanEval/95 (`check_dict_case`), where Python's `all()` short-circuits a generator expression, triggering `GeneratorExit` (a `BaseException` subclass used for control flow, not a real error). The tracer marks this as `[ERROR]`, so the model sees 7 lines of `[ERROR] GeneratorExit` and reasonably predicts fail. With only 2 such samples in the entire dataset, the model has no training signal to learn the benign nature of `GeneratorExit`. This confirms 99.83% is the theoretical ceiling for exception-only representation on this dataset.

4.2 Information Density Drives Trace Compression Benefit

Full traces improve over the no-trace baseline (+6.6pp), but VOTA achieves a much larger gain (+24.3pp). What accounts for VOTA's advantage? We decompose the gap using a label-only loss ablation: training on full traces but masking loss to the verdict label token only, eliminating gradient dilution from non-label tokens. This label-only variant achieves 86.22%, a +4.5pp improvement over standard full-trace training (81.76%). The decomposition yields:

- Gradient dilution accounts for 26% of the full-trace-to-VOTA gap (4.5pp out of 17.6pp).
- Information overload accounts for 74% (13.0pp), resolvable only through trace compression.

Even with oracle loss masking that completely eliminates gradient dilution, full traces still underperform VOTA by 13pp. The bottleneck is information density, not training signal distribution.

Table 2: Cross-model calibration for exception-only SFT. Accuracy is universally high, but calibration quality is model-dependent. All models improve over no-trace baselines.

Model	Accuracy	ECE	Brier	ECE ratio vs. no-trace
Qwen-7B	99.83%	0.0027	0.0013	63×
Qwen-1.5B	99.83%	0.0041	0.0019	∼51×
DeepSeek-6.7B	99.33%	0.0523	0.0318	∼3.4×
CodeLlama-13B	99.83%	0.0183	0.0072	∼9.3×

Table 3: SFT vs. frontier model baselines. SFT achieves the highest accuracy at ∼300× lower cost. Frontier models lack logit access for calibration. Exception-only input consistently produces the largest accuracy gain across all methods.

Method	Accuracy	Cost/1190	Latency	ECE
SFT exc. (Qwen-7B)	**99.83%**	$0.03	∼50ms	0.0027
GPT-4o 3-shot + exc.	97.2%	∼$15	∼2s	N/A
Claude 3-shot + exc.	96.5%	∼$10	∼1.8s	N/A
GPT-4o 0-shot + exc.	95.5%	∼$12	∼2s	N/A
GPT-4o 0-shot no-trace	82.3%	∼$8	∼1.5s	N/A
SFT no-trace (Qwen-7B)	75.1%	$0.02	∼30ms	0.1715

4.3 Cross-Model Calibration

Table 2 reports calibration metrics for exception-only SFT across all four models. While accuracy is consistently high (99.33–99.83%), calibration quality varies. Qwen models achieve near-perfect calibration (ECE ≤ 0.0041), while DeepSeek shows moderate miscalibration (ECE $= 0.0523$) despite high accuracy. All models show substantial calibration improvement over no-trace baselines: the exception-to-no-trace ECE ratio ranges from 3.4× (DeepSeek) to 63× (Qwen-7B).

The calibration data reinforces exception dominance from an independent evidence dimension. Exception-only models not only predict correctly but "know they are correct": Qwen-7B's Brier score of 0.0013 is 150× lower than the no-trace Brier of 0.1972. The model-dependent calibration magnitudes (DeepSeek ECE is 19× higher than Qwen) prevent us from claiming universal near-perfect calibration, but the direction of improvement is consistent.

4.4 Few-Shot Frontier Model Baselines

Table 3 compares SFT against zero-shot and few-shot prompting with GPT-4o and Claude Sonnet. GPT-4o with 3-shot exception-only prompting achieves 97.2% accuracy, 2.63pp below SFT (99.83%). This gap is statistically significant ($n = 1{,}190$, $p = 0.001$) but small in absolute terms.

Three factors justify the SFT approach despite the narrow accuracy gap. First, **cost**: SFT inference costs approximately $0.03 per 1,190 samples versus $15 for GPT-4o, a ∼300× difference at scale. Second, **calibration**: SFT provides logit-level confidence scores (ECE $= 0.0027$) while API-based models do not expose logits, precluding probability calibration. Third, **deployment**: SFT models can run offline without API dependency.

The few-shot results also reinforce the exception dominance finding from a different angle. GPT-4o gains +13.2pp from exception traces (82.3% → 95.5% zero-shot), confirming that the exception signal is valuable regardless of whether the consumer is an SFT model or a prompted frontier model.

Table 4: Full trace budget curves across four models. Each model shows a distinct pattern. Only Qwen-7B is monotonically increasing. Accuracy is reported for seed 42.

Model	**t-256**	**t-384**	**t-512**	**t-1024**	**t-2048**	**Pattern**
Qwen-7B	77.5	84.7	93.6	96.5	97.3	gradual S-curve
Qwen-1.5B	72.6	74.5	95.6	96.6	97.9	step at 512
DeepSeek	76.4	88.3	93.9	96.5	95.0	early + tail regr.
CodeLlama	77.9	73.5	97.5	98.2	41.8	W-shape

Table 5: Seed robustness at tail-512 across four models. Exception-only seed variance is shown for comparison. Full-trace seed sensitivity scales with signal complexity.

Model	**s42**	**s123**	**s456**	**Mean**	σ	**Exc.** σ
Qwen-7B	93.6	92.4	93.1	93.0	0.6pp	0.00pp
Qwen-1.5B	95.6	90.8	94.2	93.5	2.5pp	—
DeepSeek	93.9	87.6	91.9	91.1	3.2pp	4.8pp
CodeLlama	97.5	80.9	94.3	90.9	8.7pp	0.1pp

4.5 Budget Curve Analysis: Model-Dependent Patterns

Table 4 presents the full budget curve matrix across four models. Each model exhibits a qualitatively different accuracy-vs-length pattern, and no universal phase transition exists.

Qwen-7B is the only model with a clean monotonic budget curve, showing gradual improvement from 77.5% (tail-256) to 97.3% (tail-2048) with consistent gains at each step. **Qwen-1.5B** displays a sharp step function: tail-384 provides almost no improvement over tail-256 (+1.9pp), then tail-512 jumps by +21.1pp, suggesting a critical information threshold between 384 and 512 tokens. **DeepSeek** shows early-onset improvement (tail-384 already at 88.3%, +11.9pp over tail-256) but exhibits a slight regression at tail-2048 (95.0% vs. 96.5% at tail-1024), indicating that longer context introduces noise. **CodeLlama** produces a non-monotonic W-shape: tail-384 is *worse* than tail-256 (−4.4pp), tail-512 jumps to 97.5%, and tail-2048 collapses to 41.8% (−56.4pp).

The cross-model standard deviation at tail-384 is 6.4pp with a range of 14.8pp (73.5% to 88.3%). These divergent patterns confirm that the relationship between trace length and verdict accuracy is entirely model-specific. Exception-only traces, by contrast, achieve 99.83% on all four models regardless of length budget, because the exception signal is compact enough to never require truncation.

4.6 Seed Robustness of Budget Curves

Table 5 reports tail-512 accuracy across three seeds for each model, assessing whether the single-seed budget curves are reliable. Only Qwen-7B shows robust seed consistency ($\sigma = 0.6$pp). CodeLlama's $\sigma = 8.7$pp (range: 80.9%–97.5%) makes its single-seed data unreliable.

The contrast between exception-only and full-trace seed variance is stark. CodeLlama's exception-only σ is 0.1pp while its tail-512 σ is 8.7pp, an 87× ratio. This provides additional evidence for the "simplicity enables robustness" principle: the structurally simplest signal produces the most stable training outcomes.

Table 6: Error category distribution across trace conditions (Qwen-7B). Exception-only errors are irreducible; full-trace errors are dominated by information overload.

Condition	Errors	Primary category	Actionable?
exception_only	2	GeneratorExit (100%)	No (irreducible)
VOTA	7	Mixed: GenExit/attn. dilution/ambiguous	Partial
full_trace	217	Long-trace overload (65%)	No[1]
no_trace	296	Subtle bugs from code alone (37%)	No (inherent)

Table 7: Learning curve milestones (Qwen-7B). Exception-only at step 50 already surpasses full-trace's converged accuracy.

Condition	Step 50	Step 100	Step 200	Final	Conv.	Step σ
exception_only	96.2%	99.1%	99.8%	99.83%	100	0.003
full_trace	72.1%	76.8%	81.2%	81.8%	200	0.018
no_trace	71.5%	73.8%	75.0%	75.1%	150	—

4.7 Error Analysis

Table 6 breaks down prediction errors by trace condition. The error profiles are qualitatively different across conditions, confirming that different representations fail on orthogonal dimensions.

Exception-only's 2 errors are `GeneratorExit` false negatives, fully characterized in Section 4.1. VOTA's 7 errors split into three categories: 2 inherited `GeneratorExit` cases, 3 caused by loop/ae tokens diluting attention on the exception signal, and 2 from ambiguous conflicting signals (normal loops but abnormal exceptions). Full-trace's 217 errors are dominated by long-program information overload (142/217, 65%), where traces are too long for effective processing. Error samples have a median trace length 2.3× longer than correctly classified samples. The cross-condition error overlap is low (exception ∩ no-trace overlap: 0.7%), confirming orthogonal failure modes.

4.8 Training Dynamics

Table 7 compares learning curves for exception-only, full-trace, and no-trace conditions. Exception-only converges in approximately 100 training steps, reaching 99.1% accuracy. Full-trace converges more slowly (200 steps) to a much lower ceiling (81.8%). At step 50, exception-only (96.2%) already surpasses the full-trace converged accuracy, meaning exception-only reaches a higher ceiling in one-sixth of the training time. Step-to-step variance for exception-only ($\sigma = 0.003$) is 6× lower than full-trace ($\sigma = 0.018$), reflecting the smoother optimization landscape of the simpler signal.

4.9 Out-of-Distribution Generalization

Table 8 evaluates generalization to LeetCode-style problems drawn from a different distribution than the HumanEval/MBPP training data. Exception-only accuracy drops from 99.83% to 87.1% (−12.7pp), indicating partial overfitting to the training distribution's exception patterns (dominated by `TypeError`, `ValueError`, `AssertionError`). The relative ordering is preserved: exception-only (87.1%) outperforms VOTA (79.3%) and no-trace (68.2%) on OOD data, confirming that exception dominance generalizes even when absolute numbers decline.

[1] Would require architectural changes beyond LoRA SFT.

Table 8: Out-of-distribution generalization (Qwen-7B). Relative ordering is preserved, but absolute accuracy drops, especially for exception-only (−12.7pp).

Condition	In-dist.	OOD (Python)	Δ
exception_only	99.83%	87.1%	−12.7pp
VOTA	99.41%	79.3%	−20.1pp
no_trace	75.13%	68.2%	−6.9pp

Cross-language evaluation (Java, JavaScript, C++) was not feasible within the experimental timeline due to the need for language-specific trace collection infrastructure and exception format normalization. We note this as a limitation.

5 Discussion

Why simplicity is robustness. The central finding of this work is that the structurally simplest trace component is also the most robust. Exception events are short, structured, and encode a single semantic dimension: fault presence or absence. This simplicity provides robustness along every axis we measured: cross-model accuracy (99.33–99.83% on 4 models), seed stability (0.00pp σ on Qwen-7B, 0.1pp on CodeLlama), calibration (ECE as low as 0.0027), and training speed (converges in 100 steps). Every additional trace component introduces model-dependent behavior. Budget curves are the clearest example: four models produce four qualitatively different patterns, with CodeLlama exhibiting catastrophic collapse at tail-2048 (41.8%). Exception-only traces are immune to length budget effects because they never approach context limits.

VOTA as fault indicator isolation. VOTA's value lies not in generic trace compression but in automatically isolating exception events from noisy full traces. Removing assertion-point information from VOTA does not change accuracy (99.41% → 99.41%), and exception-only (99.83%) exceeds full VOTA. The "compression helps" narrative was a stepping stone; the real mechanism is diagnostic signal extraction.

Model-dependent calibration. While all models show calibration improvement from exception traces, the magnitude varies by 19× across models (Qwen-7B ECE 0.0027 vs. DeepSeek 0.0523). DeepSeek's poor calibration despite high accuracy (99.33%) parallels a broader pattern we observe: token-level training loss and sample-level classification quality can diverge. In DeepSeek's case, two seeds with nearly identical training loss (0.1225 vs. 0.122) produce accuracy differences of 8.2pp (99.33% vs. 91.09%), with the gap concentrated in fail recall (1.0 vs. 0.705). This suggests that verdict tasks should use task-specific metrics for checkpoint selection, not standard evaluation loss.

Limitations. Our dataset covers only Python and two benchmarks (HumanEval + MBPP-sanitized). OOD generalization shows 12.7pp degradation on LeetCode-style problems, and cross-language evaluation was not feasible. The study uses only four models from two architecture families. Exception-only's near-perfect accuracy on in-distribution data partially depends on the distribution of exception types in HumanEval/MBPP. Seed sensitivity on DeepSeek (one outlier seed at 91.09%) suggests that models with less stable LoRA training landscapes may require multi-seed selection or training stabilization techniques.

6 Conclusion

We studied what trace components matter for verdict prediction across four code language models. The answer is simple: exception events are the only universally robust signal. They achieve near-perfect accuracy on every model tested, with provably irreducible residual errors. All other trace components and length-accuracy relationships are model-dependent. For practitioners building verdict prediction systems, the implication is direct: instrument for exceptions, discard everything else.

References

[1] DCSH Authors. Execution Traces for Code Generation: A Systematic Comparison. In *EMNLP Findings*, 2025.

[2] TracePile Authors. TracePile: Scaling Trace-Augmented Code Training. In *NeurIPS*, 2025.

[3] TRACED Authors. TRACED: Execution-aware Pre-training for Source Code. In *ICSE*, 2024.

[4] A. Gu, B. Rozière, H. Leather, A. Solar-Lezama, G. Synnaeve, S. I. Wang. CRUXEval: A Benchmark for Code Reasoning, Understanding and Execution. In *ICML*, 2024.

[5] CodeExecutor Authors. CodeExecutor: Learning to Execute Code with Language Models. *arXiv preprint*, 2024.